ROUND ROBIN CPU SCHEDULING

---

1. AIM

To simulate the **Round Robin (RR) CPU Scheduling Algorithm** using arrival time, burst time and a fixed time quantum, and compute:

- Completion Time (CT)
- Turnaround Time (TAT)
- Waiting Time (WT)
- Average Waiting Time (AWT)

---

2. THEORY

Round Robin Scheduling

- A **preemptive** CPU scheduling algorithm commonly used in **time-sharing systems**.
- Each process is assigned a **fixed Time Quantum (TQ)**.
- Processes are executed in **circular order**.
- If a process's burst time is greater than the time quantum, the CPU is taken away and given to the next process in the ready queue.

Advantages

- Fair scheduling — no starvation.
- Good for time-sharing systems.

Disadvantages

- Large TQ → behaves like FCFS
- Too small TQ → too many context switches

Formulae

1. **Turnaround Time (TAT)**

TAT=CT−AT

2. **Waiting Time (WT)**

WT=TAT−BT

---

3. ALGORITHM

1. Input number of processes **n**.
2. For each process:
    - Process ID
    - Arrival Time (AT)
    - Burst Time (BT)
3. Input **Time Quantum (TQ)**.
4. Maintain a **ready queue**.
5. At each time:
    - Add all processes that have arrived.
    - Dequeue the process at the front.
    - Execute for **min(BT, TQ)** time.
    - If remaining BT > 0, enqueue it again.
    - Else mark the process as completed and compute CT.
6. Compute TAT, WT and Average WT.
7. Print the results.

---

4. C PROGRAM – ROUND ROBIN USING STRUCTURES

```
#include <stdio.h>

int main()
{
   int n, tq;
   int at[10], bt[10], rt[10];
   int ct[10], tat[10], wt[10];
   int time = 0, completed = 0;
   float avgwt = 0, avgtat = 0;

   printf("Enter number of processes: ");
   scanf("%d", &n);
```

```
printf("Enter Time Quantum: ");
scanf("%d", &tq);

for (int i = 0; i < n; i++)
{
    printf("Enter AT and BT for P%d: ", i + 1);
    scanf("%d %d", &at[i], &bt[i]);
    rt[i] = bt[i];
}

while (completed < n)
{
    int executed = 0;

    for (int i = 0; i < n; i++)
    {
        if (rt[i] > 0 && at[i] <= time)
        {
            executed = 1;

            if (rt[i] > tq)
            {
                time += tq;
                rt[i] -= tq;
            }
            else
            {
                time += rt[i];
                ct[i] = time;
                rt[i] = 0;
```

```
                completed++;
            }
        }
    }

    if (!executed)
        time++;
}

printf("\nProcess AT BT CT TAT WT\n");

for (int i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];

    avgwt += wt[i];
    avgtat += tat[i];

    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
           i + 1, at[i], bt[i], ct[i], tat[i], wt[i]);
}

printf("\nAverage Waiting Time = %.2f", avgwt / n);
printf("\nAverage Turnaround Time = %.2f\n", avgtat / n);

return 0;
}
```

Sample Input

Enter number of processes: 4

Enter Time Quantum: 2

Enter AT and BT for P1: 0 5

Enter AT and BT for P2: 1 4

Enter AT and BT for P3: 2 6

Enter AT and BT for P4: 3 8

Process	AT	BT	CT	TAT	WT
P1	0	5	17	17	12
P2	1	4	12	11	7
P3	2	6	19	17	11
P4	3	8	23	20	12

Average Waiting Time = 10.50

Average Turnaround Time = 16.25

=== Code Execution Successful ===